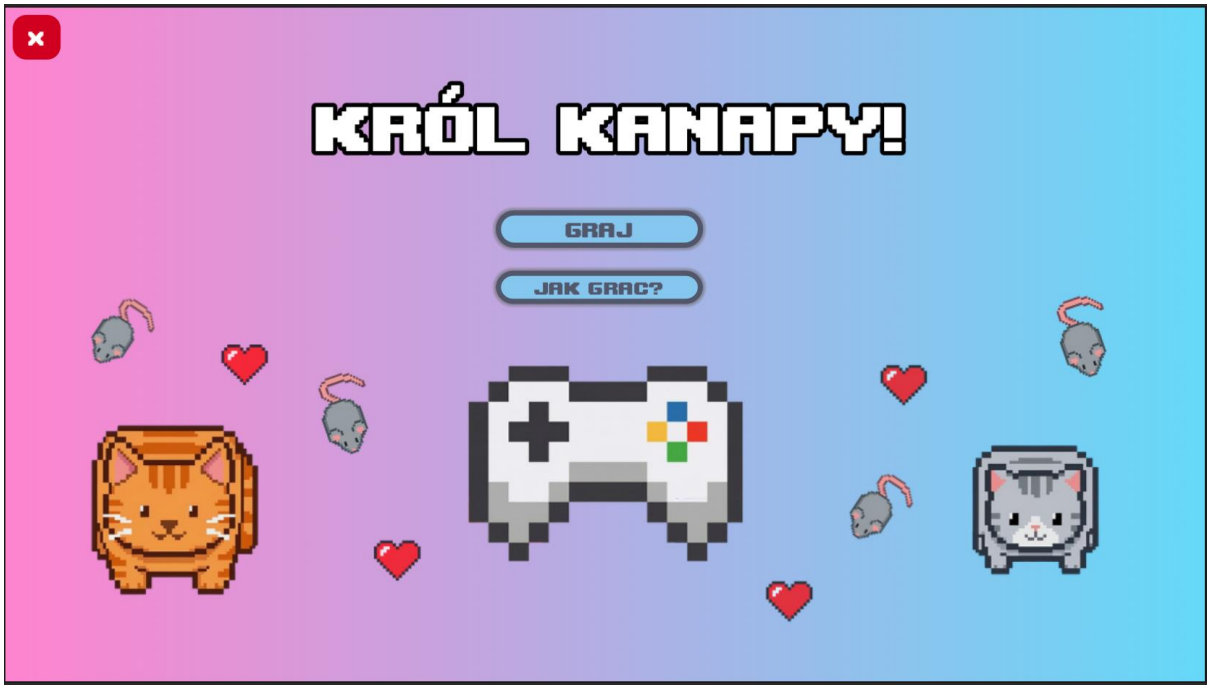
# Game Design Document: “Król Kanapy”

**Autor:** Michalina Gadomska

**Wersja:** 1.1

**Platforma:** Web (HTML5) / PC

**Silnik:** Godot Engine 4.3




## 1. Wizja Gry

- ### Koncepcja

"Król Kanapy" to autorska wersja klasycznej gry arcade "Snake". Gracz steruje Mamą Kotką, która biega po domu i ogrodzie, żeby uratować swoje małe kotki (w grze wyglądające jak myszki). Każdy zebrany kotek przyczepia się do Mamy, tworząc coraz dłuższy ogon.

- ### Grupa docelowa (Target Audience)

Gracze casualowi, fani gier retro oraz miłośnicy kotów. Gra dobra dla wszystkich.

**PREZENTACJA GRY:**

FILMIK

## 2. Mechanika Rozgrywki (Gameplay)

- ### Główna Pętla (Core Loop)

1. Gracz porusza się Mamą Kotką po mapie.
2. Zbiera "Myszki" (znajdźki), które dodają segment (Małego Kotka) do ogona.
3. Każdy segment utrudnia manewrowanie i zwiększa ryzyko kolizji.
4. Po zebraniu wymaganej liczby kotków następuje awans do kolejnego poziomu.

- ### Mechanika "Snake" i Historia Pozycji

To najważniejszy element techniczny. Segmenty ogona nie używają wbudowanych algorytmów śledzenia (Pathfinding), lecz kopiują historyczne współrzędne głowy.

- Kod zapisuje wektor pozycji `Vector2` do tablicy przy każdym ruchu.
- Każdy segment `i` ustawia swoją pozycję na `tablica[indeks - i]`.

- ### System Kolizji i Fizyki

Zastosowano precyzyjne rozdzielenie interakcji za pomocą **Collision Layers**:

- **Layer 1 (Gracz):** Reaguje na wszystko.
- **Layer 2 (Środowisko):** Ściany i meble blokują ruch.
- **Layer 3 (Przeciwnik):** Pies – posiada skrypt patrolujący; kontakt Mamy Kotki z nim natychmiast kończy grę.
- **Layer 4 (Ogon):** Specjalna warstwa wykrywana tylko przez głowę gracza. Pozwala to na "przenikanie" psa przez ogon, co zapobiega frustrującym i niesprawiedliwym przegranym.

## 3. Manager Gry (Game Management)

- ### System Poziomów

Gra składa się z 4 poziomów o rosnącym stopniu skomplikowania:

- **Poziom 1(Salon):** Dużo otwartej przestrzeni, statyczne przeszkody.
- **Poziom 2(Kuchnia):** Otwarta przestrzeń z kilkoma meblami.
- **Poziom 3 (Łazienka):** Więcej przeszkód i cięższe manewry.
- **Poziom 4 (Ogród):** Wprowadzenie Psa.

- ### Zarządzanie Czasem (Timer)

Wprowadzono element presji (Time Pressure):

- **Lvl 1-3:** 60 sekund.
- **Lvl 4:** 80 sekund (najtrudniejszy level).

Brak czasu wywołuje funkcję `koniec_gry()`.

## 4. Oprawa Wizualna i Dźwiękowa

- ### Stylistyka (Art Style)

Wykorzystano estetykę **Pixel Art**. Grafika została przygotowana w sposób spójny, łącząc autorskie generacje AI z darmowymi assetami.

- ### Sound Design

- **SFX:** Krótkie sygnały dźwiękowe dla kluczowych zdarzeń (miauknięcie przy zebraniu myszy (własnoręcznie stworzony dźwięk), szczekanie przy zbliżeniu psa, dźwięk uderzenia w ścianę).
- **Feedback:** Dźwięk służy jako natychmiastowa informacja zwrotna dla gracza o sukcesie lub błędzie.

## 5. Analiza Użycia Narzędzi AI

Projekt powstał przy wsparciu modelu językowego Gemini w następującym zakresie:

1. **Algorytmika:** Rozwiązanie problemu "bezpiecznego spawnowania" myszek. AI pomogło stworzyć pętlę `while`, która za pomocą `intersect_point` sprawdza, czy wylosowane miejsce nie koliduje z meblami.
2. **Architektura Fizyki:** Konfiguracja masek kolizji tak, aby segmenty ogona były "przezroczyste" dla psa, ale "twarde" dla gracza.
3. **Debugowanie:** Pomoc w identyfikacji błędów związanych z `z_index` (warstwy wyświetlania).

## 6. Podsumowanie Wymagań

| Kategoria | Element |
|---|---|
| **Interfejs** | Menu, Ekran Wygranej, Ekran Przegranej, Licznik |
| **Manager Gry** | System punktów, Timer (odliczanie) |
| **Fizyka** | Zaawansowane kolizje (Layers/Masks) |
| **Assety** | Grafika (AI/Licencja), Dźwięki (SFX) |
| **Sterowanie** | Klawiatura (Arrows) |
| **Mechanika** | Dynamiczny ogon, Znajdźki (Myszki) |